Assignment 5: Assignments on Functions

Name: Rajesh Mahendra Anerao

Sub: Python for Engineers (PFE)

Roll No.: 15

Branch: CSE-Data Science Engineering

Q 1.

1. **Problem Statement:** Function to check if a number is even or odd

```
6.py > number
def check_even_odd(num):
    if num % 2 == 0:
        print(f"{num} is an even number.")
    else:
        print(f"{num} is an odd number.")

number = int(input("Enter a number: "))
check_even_odd(number)
```

Output:

```
TERMINAL    PORTS    OUTPUT

PS C:\Users\Admin\OneDrive\Desktop\python 15> & C:/User
Enter a number: 20
20 is an even number.
PS C:\Users\Admin\OneDrive\Desktop\python 15>
```

Conclusions:

Determines whether a number is even or odd by checking divisibility by 2. It introduces basic conditional logic and arithmetic operations used frequently in programming tasks.

Q 2.

1. **Problem Statement:** Function to find the maximum of two numbers

```
8.py > ...
def check_maximum(num1, num2):
    if num1 > num2:
        print(f"{num1} is the maximum number.")
    elif num2 > num1:
        print(f"{num2} is the maximum number.")
    else:
        print("Both numbers are equal.")
number1 = int(input("Enter the first number: "))
number2 = int(input("Enter the second number: "))
check_maximum(number1, number2)
```

Output:

```
PS C:\Users\Admin\OneDrive\Desktop\python 15> & C:/Us
Enter the first number: 2
Enter the second number: 5
5 is the maximum number.
PS C:\Users\Admin\OneDrive\Desktop\python 15>
```

Conclusions:

Compares two numbers and returns the greater one. It demonstrates decision-making using conditions, which is essential for solving problems involving comparisons and logical operations.

Q 3.

Problem Statement: Function with keyword arguments

```
9.py > ...
def display_student_info(name, age, grade):
    print(f"Name: {name}")
    print(f"Age: {age}")
    print(f"Grade: {grade}")
name = input("Enter the student's name: ")
age = int(input("Enter the student's age: "))
grade = input("Enter the student's grade: ")
display_student_info(name, age, grade)
```

Output:

```
PS C:\Users\Admin\OneDrive\Desktop\python 15> &
Enter the student's name: Rajesh
Enter the student's age: 18
Enter the student's grade: A
Name: Rajesh
Age: 18
Grade: A
PS C:\Users\Admin\OneDrive\Desktop\python 15>
```

Conclusion:

Allows passing arguments by name instead of position. This improves readability, flexibility, and reduces errors when working with functions that have multiple parameters.

Q 4.

Problem Statement: Function with default arguments

```
10.py > ...
def greetings(name):
    print(f"Hello {name}, Welcome to VIT Pune")
greetings("NIRAJ")
```

Output:

```
TERMINAL    PORTS    OUTPUT

PS C:\Users\Admin\OneDrive\Desktop\python 15> & C
Hello NIRAJ, Welcome to VIT Pune
PS C:\Users\Admin\OneDrive\Desktop\python 15>
```

Conclusion:

Provides default values for parameters if no arguments are given. This makes functions easier to use and more flexible by reducing the need to pass all values explicitly.

Q 5.

Problem Statement: Function to Find the Factorial of a Number Using Recursion

```
11.py > ...
def find_factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * find_factorial(n - 1)
number = int(input("Enter a number to find its factorial: "))
result = find_factorial(number)
print(f"The factorial of {number} is {result}.")
```

Output:

```
PS C:\Users\Admin\OneDrive\Desktop\python 15> & C:
Enter a number to find its factorial: 12
The factorial of 12 is 479001600.
PS C:\Users\Admin\OneDrive\Desktop\python 15>
```

Conclusion:

Calculates factorial using recursion by breaking the problem into smaller steps until reaching a base case. It helps understand recursive thinking and function self-calling behavior.

Q 6.

Problem Statement: Function to Find the Sum of the Digits of the Number Recursively

```
12.py > ...
def sum_digits(num):
    if num==0:
        return 0
    else:
        return num % 10 + sum_digits(num // 10)
number = int(input("Enter a number: "))
result = sum_digits(number)
print(f"The sum of the digits in {number} is {result}.")
```

Output:

```
PS C:\Users\Admin\OneDrive\Desktop\python 15> & C:
Enter a number: 1245
The sum of the digits in 1245 is 12.
PS C:\Users\Admin\OneDrive\Desktop\python 15>
```

Conclusion:

Uses recursion to add digits of a number by repeatedly extracting the last digit. It simplifies problems involving repeated breakdown of numbers into smaller parts.

Q 7.

Problem Statement: Create a function named count vowels that accepts a string and returns the number of vowels in the string.

```
34.py > ...
def count_vowels(s):
    vowels = "aeiouAEIOU"
    count = 0
    for char in s:
        if char in vowels:
            count += 1
    return count

print(count_vowels('Hello World'))
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\Desktop\Assignment & Notes\Python> & C:/Users/
thon.exe "d:/Desktop/Assignment & Notes/Python/34.py"
3
PS D:\Desktop\Assignment & Notes\Python>
```

Conclusion:

Counts vowels in a string by iterating through characters and checking conditions. It builds understanding of string handling and basic text processing techniques.

Q 8.

Problem Statement: Write a function greet user that takes a user's name as a parameter and prints a personalized greeting message.

```python
def greet_user(name):
    print(f"Hello, {name}! Welcome to python learning.")
user_name = input("Please enter your name: ")
greet_user(user_name)
```

Output:

```
TERMINAL    PORTS    OUTPUT

PS C:\Users\Admin\OneDrive\Desktop\python 15> & C:/U
Please enter your name: Rajesh
Hello, Rajesh! Welcome to python learning.
PS C:\Users\Admin\OneDrive\Desktop\python 15>
```

Conclusion:

Accepts a user's name and prints a personalized message. It demonstrates how functions can interact with users and produce dynamic, customized outputs.

Q 9.

Problem Statement: Given a sequence of integers that end with a 00. Print the sequence in reverse order. Don't use lists or other data structures. Use the force of recursion instead.

```python
13.py > ...
def reverse_sequence():
    n = int(input("Enter a number: "))

    if n==0:
        return
    else:
        reverse_sequence()
        print(n, end=' ')
reverse_sequence()
```

Output:

```
PS C:\Users\Admin\OneDrive\Desktop\python 15> & C:/Us
Enter a number: 4
Enter a number: 5
Enter a number: 6
Enter a number: 23
Enter a number: 0
23 6 5 4
PS C:\Users\Admin\OneDrive\Desktop\python 15>
```

Conclusion:

Reverses a sequence using recursion without extra storage. It prints values during function return, illustrating recursive call flow and stack behavior effectively.